BẢN TỔNG HỢP VÀ PHÂN TÍCH CHUYÊN SÂU PAPER

ReAct: Synergizing Reasoning and Acting in Language Models

Tác giả: Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao

Princeton University & Google Research — Hội nghị **ICLR 2023**

Mã định danh: arXiv:2210.03629 | Mã nguồn: https://github.com/ysymyx/ReAct

---

1. Bối Cảnh Ra Đời & Nỗi Đau Cốt Lõi Của Các Mô Hình Trước Đó

Trước khi bài báo ReAct được công bố tại ICLR 2023, cộng đồng nghiên cứu AI chia việc ứng dụng Mô hình Ngôn ngữ Lớn (LLM) thành hai nhánh tách biệt:

1. **Trường phái Suy luận thuần túy (Reasoning-only - Chain-of-Thought / CoT):** Cho phép LLM tự "nói chuyện một mình" (internal thought traces) để chia nhỏ bài toán phức tạp thành các bước suy nghĩ trung gian (Wei et al., 2022).
2. **Trường phái Hành động thuần túy (Acting-only - Action Generation):** Cho phép mô hình sinh ra chuỗi hành động tương tác với môi trường bên ngoài (như gọi API, bấm nút trên web, gõ lệnh shell).

LÝ THUYẾT CỐT LÕI: Điểm Mù Cốt Tử Của Hai Trường Phái Cũ

Bài báo của Yao et al. đã chỉ ra điểm mù chí mạng của từng cách tiếp cận:

- **Khuyết tật của CoT thuần túy (Mô hình đóng):** CoT chỉ dựa vào tri thức tĩnh đã lưu trong trọng số (parametric memory). Do không thể tương tác với thế giới bên ngoài, mô hình không thể cập nhật thông tin thời gian thực, dễ bị **ảo giác (Hallucination)** và khi bước suy luận thứ nhất sai, sai số sẽ nhân lên theo cấp số nhân ở các bước sau (*Error Propagation*).
- **Khuyết tật của Action thuần túy (Mô hình mù):** Khi chỉ phát ra hành động (ví dụ: `Search[X]` → `Search[Y]`), mô hình hành xử như một bộ phản xạ cơ học không có trí nhớ làm việc (Working Memory). Nó không thể tự nhắc nhở mình đang tìm kiếm cái gì, không thể trừu tượng hóa các mục tiêu phụ và bất lực khi gặp lỗi ngoại lệ.

---

2. Hình Thức Hóa Toán Học Của Khung Phương Pháp ReAct

ReAct giải quyết vấn đề bằng cách **hợp nhất không gian suy luận và không gian hành động** vào một chuỗi biểu diễn ngôn ngữ thống nhất.

MÔ HÌNH HÓA TOÁN HỌC: Định Nghĩa Không Gian Trạng Thái & Hành Động Mở Rộng

Xét một bài toán Agent tương tác với môi trường:

- Tại mỗi bước thời gian t, Agent nhận được ngữ cảnh lịch sử:

$$c_t = (x, a_1, o_1, a_2, o_2, \ldots, a_{t-1}, o_{t-1}) \tag{1}$$

  trong đó x là câu hỏi/nhiệm vụ ban đầu, a_i là hành động đã thực hiện, và o_i là quan sát nhận về từ môi trường.

- Trong các kiến trúc cũ, không gian hành động $\mathcal{A}$ chỉ chứa các thao tác môi trường thuần túy. ReAct mở rộng không gian hành động thành:

$$\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L} \tag{2}$$

  trong đó $\mathcal{L}$ là không gian các biểu thức ngôn ngữ tự do đại diện cho **Ý nghĩ nội tại (Thought traces)**.

Một hành động $a_t \in \mathcal{L}$ (Thought) có đặc điểm:

- **Không thay đổi trạng thái môi trường ngoại vi** (không tốn tài nguyên gọi API, không ghi dữ liệu vào CSDL).
- **Cập nhật trạng thái bộ nhớ làm việc** của Agent: $c_{t+1} = (c_t, a_t)$, giúp định hướng cho hành động tương tác ngoại vi tiếp theo $a_{t+1} \in \mathcal{A}$.

Chu trình điều phối 3 bước tuần hoàn của ReAct:

$$\text{Thought}_t \sim \pi_\theta(\cdot \mid c_t) \quad \longrightarrow \quad \text{Action}_t \sim \pi_\theta(\cdot \mid c_t, \text{Thought}_t) \quad \longrightarrow \quad \text{Observation}_t \sim \text{Env}(\text{Action}_t) \tag{3}$$

—

3. Bốn Chức Năng Sư Phạm Của "Thought" Trong Thực Tế

Qua phân tích định tính, Yao et al. chỉ ra rằng các Thought traces sinh ra đóng 4 vai trò kiến trúc sống còn:

1. **Phân rã mục tiêu (Decomposing subgoals):** Chia câu hỏi lớn thành các câu hỏi nhỏ tuần tự (ví dụ: *"Để biết ông A hơn ông B bao nhiêu tuổi, trước hết tôi phải tra năm sinh của ông A"*).
2. **Chắt lọc tri thức (Extracting salient parts):** Khi một lệnh tìm kiếm trả về một trang web dài hàng nghìn từ, Thought đóng vai trò bộ lọc rút trích thông tin quan trọng nhất để đưa vào ngữ cảnh.
3. **Lập luận thông thường (Common-sense reasoning):** Bổ sung các phán đoán logic mà công cụ tìm kiếm không trả về trực tiếp.
4. **Tự điều chỉnh khi gặp lỗi (Exception handling & Plan shifting):** Khi một lệnh gọi Tool trả về kết quả rỗng (ví dụ: `Not found`), Thought sẽ nhận diện thất bại và đổi từ khóa tìm kiếm thay vì tiếp tục đâm đầu vào ngõ cụt.

—

4. Kết Quả Thực Nghiệm Đột Phá

Bài báo đánh giá ReAct trên 2 miền bài toán độc lập:

THỰC NGHIỆM ĐỐI CHỨNG: 1. Bài Toán Lập Luận Chuyên Sâu (HotpotQA & FEVER)

Trên bài toán trả lời câu hỏi đa bước (HotpotQA) và thẩm định tính xác thực (FEVER) tương tác với Wikipedia API:

Phương Pháp	Tương Tác API	Độ Chính Xác (F1 / Acc)	Tỷ Lệ Ảo Giác (Hallucination)
Standard Prompting	Không	Thấp	Rất cao
Chain-of-Thought (CoT)	Không	Trung bình (nhiều suy luận bịa)	Cao (56% lỗi do ảo giác)
Action-only (Act)	Có (Search/Lookup)	Thấp (thiếu lập luận)	Thấp
ReAct	**Có**	**Vượt trội**	**Giảm mạnh (chỉ 14%)**
ReAct + CoT (Hybrid)	**Có**	**Đạt điểm cao nhất**	**Tối ưu toàn diện**

Kết luận đắt giá: CoT có độ linh hoạt suy luận cao nhưng tỷ lệ bịa đặt thông tin rất lớn; Act không bịa nhưng dễ mất phương hướng. ReAct kết hợp điểm mạnh của cả hai: Vừa hành động kiểm chứng thực tế, vừa giữ vững la bàn suy luận.

THỰC NGHIỆM ĐỐI CHỨNG: 2. Bài Toán Ra Quyết Định Môi Trường (ALFWorld & WebShop)

Trên các môi trường tương tác phức tạp:

- **ALFWorld (Thao tác trong nhà dạng văn bản):** ReAct đạt tỷ lệ thành công tuyệt đối vượt qua các mô hình Học Tăng Cường (Reinforcement Learning) và Bắt chước (Imitation Learning) đến **+34%** (từ 45% lên 79%) chỉ với 1-2 ví dụ Few-Shot in-context.
- **WebShop (Mô phỏng mua sắm thương mại điện tử):** ReAct vượt qua các baseline RL **+10%** về tỷ lệ tìm đúng sản phẩm và hoàn tất giỏ hàng.

—

5. Ánh Xạ Trực Tiếp Vào Dự Án Office Asset Agent

Trong hệ thống `office-asset-agent`, chu trình ReAct được cài đặt chuẩn xác trong module `src/agents/inventory_agent.py` và `src/agents/orchestrator.py`:

```
# MINH HOA LUONG THUC THI REACT TRONG HE THONG QUAN LY VAT TU:
# 1. NGUOI DUNG HOI: "Phong Dao tao con may chiec laptop Dell va tong gia tri bao nhieu?"

# [THOUGHT 1]: Nguoi dung can biet so luong va tong gia tri cua danh muc "laptop"
# thuoc thuong hieu "Dell" tai phong ban "Phong Dao tao".
# Toi can goi Tool Deterministic SQL Calculator de tinh toan chinh xac thay vi tinh nham.

# [ACTION 1]: query_inventory_stats(category="laptop", department="Dao tao", brand="Dell")
```

```
9
10 # [OBSERVATION 1]: {"total_count": 5, "in_stock": 2, "assigned": 3, "
       total_valuation": 75000000.0}
11
12 # [THOUGHT 2]: Co so du lieu tra ve co tong cong 5 chiec, trong do 2 chiec con
       trong kho,
13 # 3 chiec dang duoc su dung. Tong tri gia la 75 trieu dong. Du lieu da day du de
       tra loi.
14
15 # [FINAL RESPONSE]: "Thua quy thay co, tai Phong Dao tao hien co tong cong 5 chiec
       laptop Dell
16 # voi tong gia tri la 75.000.000 dong (trong do 2 chiec dang san co trong kho)."
```

—

6. Bộ Câu Hỏi Phản Biện Chuyên Sâu Của Hội Đồng (Defense Insights)

ỨNG DỤNG VÀ PHẢN BIỆN: 3 Câu Hỏi Đắt Giá Nhất Về ReAct Mà Hội Đồng Sẽ Hỏi Bạn

1. **Hội đồng hỏi:** *"Tại sao ReAct lại tốn nhiều token hơn CoT thông thường? Em đã tối ưu hóa chi phí token như thế nào?"*
   Đáp án chuẩn: ReAct tốn thêm token do phải lưu trữ cả các chuỗi Thought trung gian và văn bản Observation trả về từ Tool. Trong đồ án, em đã tối ưu bằng cách: Chỉ lưu các trường tóm tắt có cấu trúc của Tool (JSON compact) thay vì nhét toàn bộ HTML/văn bản thô, và thiết lập cơ chế cắt tỉa ngữ cảnh (Context Pruning) loại bỏ các bước tìm kiếm trung gian sau khi đã trích xuất được kết quả.

2. **Hội đồng hỏi:** *"Nếu Action trả về kết quả rỗng (Observation = None) thì ReAct có bị rơi vào vòng lặp vô tận không?"*
   Đáp án chuẩn: Nếu không kiểm soát, ReAct có thể liên tục sinh Thought và thử lại. Em đã giải quyết bằng cách áp dụng **Ngưỡng thử lại tối đa (Max Retries = 3)** và cơ chế ngắt mạch (Circuit Breaker) trong LangGraph: Sau 3 lần quan sát rỗng, Agent bắt buộc phải chuyển sang bước kích hoạt Fallback hỏi lại người dùng.

3. **Hội đồng hỏi:** *"Sự khác biệt bản chất giữa ReAct và OpenAI Function Calling là gì?"*
   Đáp án chuẩn: Function Calling ban đầu chỉ là cơ chế ánh xạ từ văn bản sang cú pháp JSON của hàm (Action-only). ReAct là một **khung nhận thức toàn diện** bao bọc bên ngoài: Nó bắt buộc mô hình phải sinh ra lý do suy luận (Thought) trước khi quyết định gọi hàm, giúp việc chọn hàm chính xác hơn và cho phép con người dễ dàng đọc vết giải trình (Explainability).